



Research Intake 2026

Day 12

TEXT PREPROCESSING & FEATURE ENGINEERING

A Beginner's Guide for Students

From foundational concepts to research-level understanding

Gudsky Research Foundation

Table of Contents

- 1 Why Preprocessing Matters — Garbage in, garbage out: the quality foundation
 - 1.1 Impact on Model Performance
 - 1.2 The Preprocessing–Overfitting Trade-off
- 2 Text Cleaning Techniques — Noise removal, normalisation, and standardisation
 - 2.1 Lowercasing and Unicode Normalisation
 - 2.2 Punctuation and Special Character Removal
 - 2.3 Number Handling
 - 2.4 HTML and URL Stripping
- 3 Tokenisation In Depth — Word, sentence, subword, and character-level methods
 - 3.1 Whitespace vs. Rule-Based Tokenisers
 - 3.2 Byte-Pair Encoding (BPE) and WordPiece
 - 3.3 Language-Specific Challenges
- 4 Stop Word Removal — When to remove and when to keep
 - 4.1 Standard Stop Word Lists
 - 4.2 Domain-Specific Stop Words
- 5 Stemming and Lemmatisation — Reducing words to canonical forms
 - 5.1 Porter and Snowball Stemmers
 - 5.2 WordNet Lemmatiser
 - 5.3 When Each Method Wins
- 6 Bag of Words Representation — From tokens to feature vectors
 - 6.1 Count Vectors and Sparse Matrices
 - 6.2 Vocabulary Size and Dimensionality
- 7 TF-IDF: Theory and Practice — Weighting by importance, not just frequency
 - 7.1 Term Frequency and Inverse Document Frequency
 - 7.2 Variants: Sublinear TF, Smooth IDF
 - 7.3 TF-IDF in Production Search Engines
- 8 Advanced Feature Engineering — Going beyond bag-of-words
 - 8.1 N-Gram Features
 - 8.2 Character N-Grams for Robustness
 - 8.3 Linguistic Features: POS, Dependency, NER
- 9 Research Perspectives — Learned preprocessing vs. manual pipelines

1

Why Preprocessing Matters

Garbage in, garbage out — the quality foundation

Text in the wild is profoundly messy. Real-world corpora contain HTML tags, emoji, inconsistent capitalisation, misspellings, URLs, boilerplate navigation text, encoding artefacts, and an endless variety of abbreviations — all of which masquerade as meaningful tokens if left unaddressed. A model trained on uncleaned text learns noise as signal, wasting representational capacity on surface variation that carries no semantic content. It will treat 'apple', 'Apple', 'APPLE', and 'Apple®' as four entirely different entities, inflating vocabulary size and diluting the statistical evidence for each meaningful token.

The relationship between preprocessing and model quality is not simply 'more cleaning is better'. Conversely, over-aggressive cleaning destroys semantically important information: removing all punctuation strips the exclamation marks and question marks that are primary signals for sentiment and intent; removing all numbers eliminates the dates, prices, and quantities that drive financial and clinical NLP tasks; stripping all capitalisation conflates 'US' (United States) with 'us' (first person plural pronoun) and 'IT' (information technology) with 'it' (the pronoun). The practitioner's goal is targeted cleaning — removing genuine noise while preserving genuine signal.

The principle that preprocessing strategy must be task-specific cannot be overstated. A preprocessing pipeline appropriate for a topic modelling task — aggressive stopword removal, heavy stemming, no punctuation — will actively harm a sentiment analysis system, which needs exclamation marks, negations, and intensifiers. A pipeline appropriate for social media classification — preserving hashtags, emoji, and slang — will add noise to a medical note extraction system. This section establishes the theoretical framework for making these decisions systematically rather than by trial and error.



Use Case: Spam Detection Failure Mode

An early Bayesian spam filter that stripped punctuation failed to block messages like 'V.I.A.G.R.A' because the deliberately garbled word did not appear in its vocabulary. The adversarial insertion of punctuation into words is a classic evasion technique that exploits exactly this preprocessing vulnerability. Character-level n-gram features that survive punctuation stripping would have caught it — demonstrating that preprocessing choices directly shape a model's adversarial robustness, not just its accuracy on clean test data.

1.1 Impact on Model Performance

The empirical evidence for preprocessing's impact is substantial and well-documented. Studies on the Reuters-21578 news classification benchmark — a standard evaluation corpus containing 21,578 news wire articles across 135 categories — consistently show 3–8% accuracy improvements from careful preprocessing over raw text input. The effect is systematically larger for classical machine learning methods (TF-IDF + SVM, Naive Bayes, Logistic Regression) than for modern transformer-based architectures, for an important structural reason: transformers have learned their own internal normalisation through subword tokenisation and massive pre-training, while classical models have no such implicit compensation mechanism.

For classical models, preprocessing directly controls vocabulary size, which has second-order effects on everything downstream. A raw vocabulary might contain 150,000 tokens for a medium-sized corpus; after lowercasing, stemming, and stopword removal, it might shrink to 20,000. This fivefold reduction means the TF-IDF matrix has five times fewer columns, enabling faster training, lower memory consumption, and — critically — better statistical estimates for each feature because the same amount of training data is now spread across fewer tokens. The bias-variance trade-off directly favours the smaller vocabulary.

For transformer models, the calculus is different. BERT, RoBERTa, and their successors use WordPiece or BPE tokenisation that already handles many surface variation problems (capitalisation variants, morphological forms) through shared subword pieces. Pre-training on hundreds of billions of tokens means these models have already seen enough variation to generalise across many surface forms. However, this does not mean preprocessing is irrelevant for transformers: removing HTML tags, deduplicating training data, and domain-specific cleaning of clinical or legal text can still yield meaningful performance improvements, particularly when fine-tuning on small target datasets where the model has limited opportunity to learn from in-domain noise.

1.2 The Preprocessing–Overfitting Trade-off

Aggressive normalisation reduces vocabulary size and thereby limits a model's capacity to overfit to rare surface forms. A model that sees 'running', 'runs', 'ran', and 'runner' as four distinct tokens can potentially overfit to the specific distribution of these forms in the training set; a model that sees them all as 'run' (after lemmatisation) is protected from this particular failure mode. This is the classic bias-variance trade-off manifesting at the feature engineering level: normalisation introduces bias (treating morphologically distinct forms as identical) in exchange for reduced variance (less sensitivity to training set peculiarities).

The counter-argument is equally important: sentiment classifiers benefit from preserving 'not', which transforms the meaning of the words that follow it. A model that lemmatises 'not good' and 'not bad' both to 'not good/bad' without preserving the structure of negation will fail to distinguish them correctly. Similarly, preserving '!!!' signals high-intensity sentiment that plain word tokens do not capture. Preserving emoji — common in social media NLP — can provide strong affective signals. The practitioner must audit, for each task, which surface features carry discriminative signal and which are genuine noise, and design the preprocessing pipeline accordingly.

A practical framework for making these decisions is to build a baseline with minimal preprocessing (just lowercasing and URL removal), then ablate each additional preprocessing step — stopword removal, stemming, number stripping — measuring its effect on held-out validation performance. Steps that improve validation performance are included; steps that hurt it are excluded. This empirical approach is more reliable than general intuitions about what 'should' matter, because the optimal preprocessing is always an empirical property of the specific task, domain, and model architecture.

Text cleaning is the transformation of raw, noisy input text into a normalised form suitable for tokenisation and downstream processing. It is the first and most fundamental step in the NLP pipeline, and errors or omissions here propagate through every subsequent stage. A comprehensive text cleaning pipeline addresses five categories of noise: encoding and character set issues, surface case variation, punctuation and special characters, numerical content, and web-specific content such as HTML and URLs. The appropriate handling of each category depends on the task, domain, and model architecture.

A key engineering principle for text cleaning is to make each operation explicit, ordered, and reversible where possible. Using a sequence of specific regex substitutions — rather than a single aggressive character whitelist — makes the pipeline transparent, debuggable, and easy to modify when task requirements change. Each cleaning step should be logged and its effect on vocabulary size and downstream metric measured, so the team maintains a clear understanding of what each step contributes.

2.1 Lowercasing and Unicode Normalisation

Lowercasing is the most universally applied text cleaning operation. It collapses 'Apple', 'apple', and 'APPLE' into a single token, preventing the model from treating them as three distinct entities and tripling the statistical evidence available for the concept. For most NLP tasks, this is a straightforward improvement. However, case carries semantic information in several important contexts that practitioners must be aware of.

Proper nouns and acronyms are particularly susceptible to case-related ambiguity. 'US' means the United States; 'us' is a first-person pronoun. 'IT' in a corporate document refers to information technology; 'it' is a pronoun. 'PTSD' and 'AIDS' are medical acronyms with very different meanings from their lowercase counterparts. For NER tasks, clinical NLP, and any domain where acronyms are common and consequential, cased models — which preserve capitalisation — systematically outperform uncased models. BERT, for example, offers both 'bert-base-uncased' and 'bert-base-cased' variants, and the cased variant should be the default for named entity recognition tasks.

Unicode normalisation is a distinct operation from lowercasing and is frequently overlooked, even by experienced practitioners. The Unicode standard allows multiple byte sequences to represent what appears to be the same character — a phenomenon arising from the history of character encoding. The string 'café', for example, can be represented either as four code points (c, a, f, é in NFC form) or as five code points (c, a, f, e, combining acute accent in NFD form). These are visually identical but byte-for-byte different, causing them to be treated as different tokens by any system that does not normalise first. Python's `unicodedata.normalize('NFC', text)` resolves this systematically. Web-scraped and multilingual corpora are particularly prone to mixed normalisation forms.



Common Crawl Cleaning Pipeline

GPT-3 trained on 570 GB of filtered Common Crawl. OpenAI removed pages from known low-quality domains, applied near-duplicate detection using MinHash (a locality-sensitive hashing technique that identifies documents sharing more than ~80% of their n-grams), and trained a linear SVM on WebText — the high-quality Reddit-linked reference corpus — to score and discard low-quality pages. This multi-step cleaning pipeline at massive scale illustrates that for large language model training, data quality is at least as important as data quantity: higher-quality training data produces better models than more low-quality data, even after controlling for total token count.

2.2 Punctuation and Special Character Removal

Punctuation removal is conventionally implemented as regex substitution: replacing characters outside the set [a-zA-Z0-9] with a space or empty string. This approach is effective for topic classification and information retrieval tasks where punctuation is genuinely uninformative, but it requires careful thought for any task where punctuation carries semantic or pragmatic meaning.

The most important special cases are: hyphens in compound adjectives ('state-of-the-art', 'well-known', 'end-to-end'), which connect semantically related tokens and should ideally be preserved or replaced with a space rather than simply deleted; apostrophes in contractions ('don't', 'it's', 'I've'), which when deleted merge the two components ('dont', 'its', 'ive') into non-standard forms that may not appear in training vocabulary; currency symbols (£, €, ¥, \$), which in financial and e-commerce contexts signal monetary quantities; and email addresses and hashtags in social media corpora, which are meaningful named entities rather than noise. A task-aware whitelist of characters to preserve — rather than a blanket strip of everything non-alphanumeric — is always preferable for tasks where these distinctions matter.

For sentiment analysis specifically, punctuation is a primary carrier of emotional intensity. The difference between 'Great movie.' and 'Great movie!!!' is encoded entirely in punctuation. Sophisticated sentiment systems either preserve punctuation in a structured form (counting exclamation marks as a feature) or use specialised tokenisation that treats 'Great!' and 'Great' differently. Simply stripping all punctuation discards this signal entirely and measurably degrades sentiment classification accuracy.

2.3 Number Handling

Numerical tokens present a spectrum of processing options, each appropriate for different tasks. The four main approaches are: deletion (remove all digit sequences), substitution with a placeholder token (replace any number with the special token <NUM>), normalisation (standardise surface variants of the same quantity, e.g. '1,234', '1234', '1.2K', '1.2k' all to '1234'), and preservation (keep numbers exactly as they appear).

The choice among these options is heavily task-dependent. Financial text classifiers that delete all numbers lose the most critical signals in their domain: the difference between 'earnings rose 12%' and 'earnings rose 3%' is entirely in the numbers. Clinical NLP systems that process medical notes depend on numerical values for dosage, vital signs, and lab results; stripping numbers would catastrophically degrade their performance. By contrast, a topic classifier that categorises news articles into 'sport', 'politics', 'technology', and 'business' gains little from preserving the specific numerical values in headlines, and may benefit from vocabulary reduction through number substitution.

A particularly effective intermediate approach is semantic number normalisation: preserving the presence of numbers as a signal (via a <NUM> placeholder) while suppressing the specific values that would create vocabulary sparsity. This is especially valuable for tasks where 'the product costs <NUM>' is informative (tells the classifier this is a commerce-related text) but '1,499.99' vs '1,500.00' is not. The placeholder approach gives the model knowledge that a quantity exists without requiring it to memorise the specific values, improving generalisation to new numerical contexts.

2.4 HTML and URL Stripping

Web-scraped corpora — which include Common Crawl, Wikipedia dumps, and the majority of large-scale NLP training datasets — contain substantial quantities of HTML markup, JavaScript code, CSS, and boilerplate navigation text that is semantically uninformative for most NLP tasks. A raw HTML page scraped from a news site might contain 2,000 words of actual article content surrounded by 8,000 words of navigation menus, footer text, cookie consent notices, and inline scripts. Processing this raw HTML treats all content equally, causing the model to learn from boilerplate at the same level as article content.

The standard library for HTML cleaning in Python is BeautifulSoup (from the beautifulsoup4 package), which parses HTML and provides structured access to its components. A typical cleaning pipeline extracts text from specific HTML tags (<p> for paragraphs, <h1>–<h6> for headings, for list items) and ignores

<script>, <style>, <nav>, and <footer> content. This targeted extraction is significantly more effective than naive tag stripping with regex, which cannot handle malformed HTML or nested structures correctly.

URL handling requires a similar decision framework to number handling. URLs can be deleted entirely, replaced with a placeholder token (<URL>), partially preserved (keeping only the domain name as a feature), or parsed into their components (protocol, domain, path, query parameters) and treated as structured features. For web-genre classification — distinguishing news articles from blog posts from academic papers — the domain name is a strong feature and should be preserved. For content-based classification where the source domain is not informative, URL replacement with a placeholder token is appropriate. Leaving URLs intact creates massive vocabulary explosion as each unique URL becomes a distinct token, almost all of which appear only once in the corpus.

Tokenisation is the process of segmenting a continuous text stream into discrete units — tokens — that become the atomic inputs to subsequent processing. It is conceptually simple but technically complex: the definition of what constitutes a token is not universally agreed upon, varies across languages, and has significant downstream consequences for model vocabulary size, out-of-vocabulary rates, and ultimately task performance. Tokenisation decisions made early in a pipeline constrain every subsequent design choice.

The tokenisation problem is fundamentally a boundary detection problem: given a sequence of characters, where are the boundaries between meaningful units? For English and most Western European languages, whitespace provides a useful but imperfect signal: most word boundaries coincide with spaces, but not all spaces signal word boundaries (abbreviations, multi-word expressions) and not all word boundaries coincide with spaces (contractions, hyphenated compounds, clitics). For morphologically complex or agglutinative languages, the word boundary problem is further complicated by the fact that what appears to be a single word may encode what requires an entire phrase in English.

3.1 Whitespace vs. Rule-Based Tokenisers

Splitting on whitespace is the computationally trivial baseline tokeniser: split the string on any whitespace character and return the resulting list. This handles the majority of English text correctly but fails systematically on a predictable set of cases. Contractions are the most common: 'don't' becomes ['don't'] rather than ['do', 'n't'], meaning the negative particle is fused with the modal verb and the fused token is unlikely to appear in a model trained on data where contractions were tokenised differently. Sentence-final punctuation is the second most common problem: 'end.' becomes ['end.'], fusing the period with the word and preventing the model from recognising 'end' as the same token as 'end' in non-final position.

Rule-based tokenisers — NLTK's `word_tokenize` (based on the Penn Treebank tokenisation conventions) and spaCy's language-specific tokenisers — address these problems through comprehensive, hand-crafted exception lists and regular expression patterns. The Penn Treebank conventions, originally developed for the manually annotated Penn Treebank corpus, specify how English text should be tokenised for syntactic annotation: contractions are split into their morphological components, punctuation is separated from words, and special tokens (URLs, email addresses, dates) are handled as single units. Following these conventions ensures consistency with the training data used for pre-trained NLP models.

spaCy's tokeniser is particularly well-engineered for production use. It processes text through a sequence of steps: first splitting on whitespace, then applying a set of prefix rules (e.g., strip leading punctuation), suffix rules (e.g., strip trailing punctuation and possessives), and infix rules (e.g., split on hyphens in specific contexts). It also maintains exception dictionaries for common cases like 'U.S.A.' that should not be split and 'won't' that should be split into ['wo', 'n't']. The result is a tokenisation that handles the vast majority of English surface forms correctly without requiring task-specific customisation.

3.2 Byte-Pair Encoding (BPE) and WordPiece

Byte-Pair Encoding, introduced for machine translation by Sennrich et al. (2016) and subsequently adopted as the tokenisation algorithm for GPT-2, GPT-3, and GPT-4, addresses the fundamental limitation of word-level tokenisation: the out-of-vocabulary (OOV) problem. Any fixed vocabulary of words will encounter words in test data that were not present in training data. These OOV words are typically replaced with a special `<UNK>` token, discarding all information about their form. For a language model that must generate or understand text in open-domain settings, a significant OOV rate is catastrophic.

BPE's solution is to start with a character-level vocabulary (which has a zero OOV rate, since any word can be spelled out character by character) and iteratively merge the most frequent adjacent pair of tokens in the

training corpus. The merge process continues until the vocabulary reaches a predetermined target size (typically 32,000 to 50,000 tokens for modern LLMs). The resulting vocabulary contains a mixture of character-level units (for rare or novel words), common morphological pieces ('ing', 'tion', 'ness', 'un-'), and complete common words ('the', 'is', 'and'). Any unseen word at test time is automatically decomposed into the longest matching subword pieces, guaranteeing zero OOV rate while maintaining efficient representation for common words.

WordPiece vs BPE — Key Comparison

- BPE** (GPT-2, GPT-3, GPT-4, RoBERTa)
Merge criterion: raw frequency of adjacent pair in corpus
'unhappiness' → ['un', 'happiness'] or ['un', 'happi', 'ness']
- WordPiece** (BERT, DistilBERT, ELECTRA)
Merge criterion: maximise likelihood of training data
'unhappiness' → ['un', '##happi', '##ness'] (## = continuation)
- SentencePiece** (mBERT, XLM-R, T5, multilingual models)
Language-agnostic: treats all Unicode as character space
No whitespace assumption: works for Chinese, Japanese, Arabic
'unhappiness' → ['_un', 'happi', 'ness'] (_ = word start)

The practical differences between BPE and WordPiece are small for most tasks — both produce similar segmentations for common vocabulary and handle novel words in similar ways. The choice of algorithm is therefore primarily determined by which pre-trained model one is fine-tuning: if using a BERT-family model, use the WordPiece tokeniser included with that model; if using a GPT-family model, use its BPE tokeniser. Mixing tokenisers between pre-training and fine-tuning produces degraded performance because the model's internal representations are built around a specific vocabulary.

3.3 Language-Specific Challenges

The tokenisation strategies discussed above were developed primarily for English and other Western European languages with whitespace-delimited, morphologically moderate structure. The majority of the world's languages present tokenisation challenges that require fundamentally different approaches.

Chinese and Japanese are written without word-delimiters: characters are concatenated into a continuous stream, and word boundaries must be inferred from context. Chinese word segmentation — the task of identifying where words begin and end in a Chinese character sequence — is itself a non-trivial NLP problem, typically addressed with trained sequence labelling models. The jieba library provides a widely used Python implementation for Chinese segmentation. Japanese presents an additional complication: the writing system mixes three character sets (hiragana, katakana, and kanji), each with different tokenisation conventions.

Arabic and other Semitic scripts present a different set of challenges: the script is right-to-left, letters connect and change shape depending on their position in a word, and the language is morphologically rich — a single Arabic word can encode information that requires an entire English clause. Tokenising Arabic without morphological analysis produces tokens that represent complex morphological bundles, inflating vocabulary size and obscuring the underlying semantic units. Morphological tokenisers that decompose Arabic words into their constituent prefixes, stems, and suffixes are standard for Arabic NLP.

| Language | Challenge | Standard Approach |
|----------|--------------------------|-------------------------------------|
| Chinese | No word delimiters | jieba / PKU/MSR segmentation models |
| Japanese | Mixed scripts, no spaces | McCab, KyTea, or SentencePiece |

| | | |
|-----------------|---|--|
| Arabic | Morphological complexity, right-to-left | Farasa, Madamira morphological tokeniser |
| Thai | No word boundaries | NECTEC / PyThaiNLP |
| Finnish/Turkish | Agglutinative: 1 word = phrase | Morfessor subword segmentation |
| German | Compound nouns: Hauptbahnhof | Compound splitter (CharSplit) |

Stop words are words that appear with very high frequency across all or most documents in a corpus, contributing little discriminative power for classification or retrieval tasks while consuming a disproportionate share of vocabulary space and feature weight. The classic English stop words — 'the', 'a', 'an', 'is', 'are', 'at', 'in', 'on', 'and', 'or', 'but', 'of', 'to', 'for', 'with' — together account for 30–40% of all tokens in typical English text. Removing them reduces feature dimensionality, speeds up training and inference, and often improves model generalisation by forcing the model to rely on more informative vocabulary.

The theoretical justification for stop word removal is rooted in information theory: the information content of a word for a given classification task is inversely proportional to how evenly it is distributed across classes. A word that appears with equal probability in sports articles, financial articles, political articles, and technology articles carries zero information about which category a document belongs to. Stop words, by definition, are words that approach this uniform distribution across all topics and genres. Removing them is equivalent to discarding features with near-zero information gain — a step that all principled feature selection algorithms (chi-squared, mutual information) would also recommend.

4.1 Standard Stop Word Lists

NLTK provides 179 English stop words accessible through `nltk.corpus.stopwords.words('english')`. This list was curated to include the most frequent grammatical function words in English — articles, prepositions, conjunctions, pronouns, auxiliary verbs, and common adverbs — that carry grammatical but not lexical meaning. The list has been widely adopted as a standard baseline and is available for 22 additional languages in NLTK, including German (232 words), French (157 words), and Spanish (313 words).

spaCy integrates stop word detection directly into its linguistic annotation pipeline, marking each token with a boolean `is_stop` attribute. This approach is more convenient than post-hoc filtering because it integrates with spaCy's full pipeline — one can simultaneously filter stop words and access POS tags, dependency labels, and named entities in a single pass over the text. spaCy's stop word lists are somewhat larger than NLTK's (305 English words) and include some content words that are very high frequency in general English ('seem', 'herself', 'whatever', 'anywhere') but not necessarily in all domains.

An important practical consideration is that standard stop word lists are static and were designed for general English text. They may be inappropriate for specific domains. In a legal corpus, words like 'shall', 'herein', 'whereas', and 'notwithstanding' are very common but carry legally specific meanings and may be discriminative for sub-genre classification. In a poetry corpus, function words like 'the' and 'a' are part of the metre and should not be removed. The practitioner should review the standard list in the context of their specific task and be prepared to add exceptions.

4.2 Domain-Specific Stop Words

Beyond the standard general-language stop words, any sufficiently large and homogeneous domain corpus will contain domain-specific high-frequency words that function as stop words within that domain. These are words that are diagnostic of the domain itself but not discriminative within it — words that tell you the corpus is medical, but not which medical topic a given document addresses.

In a medical corpus, words like 'patient', 'hospital', 'treatment', 'doctor', 'clinical', and 'diagnosis' appear with such overwhelming frequency across all medical documents that they provide virtually no discriminative power for a classifier trying to distinguish between, say, cardiology notes and oncology notes. Adding these words to the stop list can substantially improve within-domain classification accuracy. A principled approach to identifying domain stop words is to compute the document frequency of each token across the

corpus and flag tokens with $df > \text{threshold}$ (e.g., appearing in more than 80% of documents) as domain-specific stop word candidates for human review.

The same principle applies in reverse: standard stop words may actually be informative within specific domains. In a corpus of financial reports, the word 'will' (a standard NLTK stop word) may discriminate between forward-looking statements and backward-looking analysis — a legally and analytically important distinction. In a corpus of question-answering pairs, interrogative words like 'who', 'what', 'where', 'when', 'why', and 'how' (all standard stop words) are precisely the discriminative features needed to identify question type. Domain context always takes precedence over the generic stop word list.

Stemming and lemmatisation are two approaches to the same underlying goal: reducing the vocabulary of a corpus by mapping morphologically related word forms to a common canonical representation. The motivation is that 'run', 'runs', 'ran', and 'running' are four distinct tokens in a raw vocabulary but represent the same underlying concept, and a model that treats them as four unrelated entities must learn their semantic relationships from scratch rather than having this knowledge encoded in the feature representation. By mapping all four to a single representation, we increase the statistical evidence available for the concept and reduce the vocabulary size, improving both sample efficiency and generalisation.

The distinction between the two approaches lies in the mechanism used to determine the canonical form and the linguistic validity of the result. Stemming uses fast, rule-based heuristics to strip suffixes, producing stems that need not be valid words. Lemmatisation uses a lexical database and often grammatical information (part of speech) to look up the true dictionary base form, always producing a valid word. The choice between them is a trade-off between speed and linguistic accuracy that must be evaluated in the context of the specific task.

5.1 Porter and Snowball Stemmers

The Porter stemmer (Martin Porter, 1980) is the most widely used English stemmer and remains a standard baseline in information retrieval after more than four decades. It operates through a cascade of five rule sets applied sequentially, each targeting a specific class of suffixes. The first rule set handles plurals and past tenses (caresses → caress, ponies → poni, cats → cat). Subsequent rule sets handle additional derivational suffixes (causal → causal, relational → relat, valenci → valenc). The cascade design means that the output of one rule set feeds into the next, allowing composite suffixes to be stripped in stages.

The Snowball stemmer — also developed by Porter as an improved successor, formally known as Porter2 — extends the approach with a more formal specification language (Snowball) and implementations for 13 languages including German, French, Spanish, Italian, Dutch, Finnish, Swedish, Norwegian, Danish, Russian, and Portuguese. NLTK provides the SnowballStemmer with language selection: SnowballStemmer('german'), SnowballStemmer('french'), and so on. For multilingual NLP pipelines, the Snowball stemmer is the standard choice for languages covered by its implementations.

The characteristic failure modes of stemmers arise from their rule-based nature: they cannot handle irregular forms (goose → goos, not gees → gees), they over-stem related-but-distinct words ('universe' and 'university' both stem to 'univers', which is linguistically incorrect and loses the semantic distinction between them), and they under-stem in cases where the suffix is ambiguous or where the root word ends in a consonant cluster that the rules cannot correctly identify. These failure modes are predictable and well-documented, making it straightforward to identify cases where stemming is inappropriate.

5.2 WordNet Lemmatiser

NLTK's WordNetLemmatizer uses WordNet — the large Princeton lexical database of English that organises words into synonym sets (synsets) connected by semantic relationships — to return the true dictionary base form (lemma) of a word. Unlike the Porter stemmer, which applies rules without consulting a dictionary, the WordNet lemmatiser performs actual lexical lookup: it searches WordNet for the word form and returns the lemma from the corresponding synset.

The critical dependency of WordNet lemmatisation is part-of-speech information. The lemma of 'meeting' depends entirely on its grammatical role: as a noun, 'meeting' lemmatises to 'meeting' (a gathering is itself the base form); as a verb, it lemmatises to 'meet' (the gerundive form of the action). Without POS specification, the lemmatiser defaults to treating every word as a noun, producing incorrect lemmas for verbs, adjectives, and adverbs. The correct usage requires obtaining POS tags from a tagger (NLTK's

averaged_perceptron_tagger or spaCy's tagger) and converting Penn Treebank POS tags to WordNet POS constants before calling the lemmatiser.

WordNet lemmatisation is 10 to 50 times slower than Porter stemming per token, depending on implementation. This performance gap is typically irrelevant for offline batch processing — the speed difference between processing a million tokens in two seconds vs. 50 seconds is rarely the bottleneck in an NLP pipeline. However, for real-time online systems (interactive search, streaming text analysis) where latency is measured in milliseconds, the speed difference may be material and stemming may be preferred. Modern deep learning pipelines increasingly bypass both approaches in favour of subword tokenisation, rendering the stemming-vs-lemmatisation choice a classical ML concern.

| Use Stemming When | Use Lemmatisation When |
|--|---|
| Speed is critical (real-time search, streaming) | Semantic precision matters (Q&A, NER, knowledge graphs) |
| Morphological variants are pure noise for the task | Generated or retrieved text must be human-readable |
| The target language lacks WordNet coverage | Building knowledge graphs or ontologies from text |
| Large vocabulary needs aggressive reduction | Task involves irregular forms (geese, better, went) |
| Prototype / baseline pipeline with tight deadline | Production system where quality trumps speed |

5.3 When Each Method Wins

The empirical evidence on stemming vs. lemmatisation is mixed and task-dependent, which is the expected result given that both methods target the same underlying goal through different mechanisms. For information retrieval tasks — where the goal is to match documents to queries based on shared content words — stemming has historically performed comparably to lemmatisation at much lower computational cost, because the imprecision of stemming (merging related but distinct concepts) is roughly offset by improved recall for morphological variants. This has made stemming the default in production search engines.

For classification and extraction tasks where semantic precision matters more than recall, lemmatisation typically outperforms stemming. The systematic errors of stemming — merging 'universe' and 'university', producing non-word stems like 'studi' for 'studies' — introduce noise that degrades classifier performance on tasks where these distinctions are relevant. Named entity recognition in particular benefits from lemmatisation because entity names and their inflected forms must be correctly identified as variants of the same entity, and stemming-induced collisions between different entities (merging different names to the same stem) introduce false positive coreferences.

The Bag of Words (BoW) model is the foundational text representation for classical machine learning approaches to NLP. It encodes each document as a vector of numerical values over a fixed vocabulary, discarding all information about word order and treating each document as an unordered multiset of tokens. Despite this severe simplification, BoW representations are surprisingly effective for many text classification tasks, particularly those where topic membership — which words appear, regardless of in what order — is the primary discriminative signal.

The BoW representation emerged from the information retrieval community in the 1950s and 1960s, where it was used to index and retrieve documents based on word occurrences. Its formal mathematical properties were studied extensively in the context of vector space models of information retrieval (Salton, 1971), and it was subsequently adopted as the standard feature representation for text classification when machine learning methods (Naive Bayes, logistic regression, SVMs) began to be applied to text in the 1990s. The model's simplicity is both its greatest weakness (it ignores structure) and its greatest strength (it scales linearly with corpus size and vocabulary).

6.1 Count Vectors and Sparse Matrices

In the standard BoW formulation, the vocabulary V is the set of all unique tokens observed in the training corpus (after whatever preprocessing has been applied). Each document d is represented as a vector of length $|V|$, where the i -th element is the count of vocabulary term v_i in document d . This representation is mathematically straightforward but has a critical practical issue: for a typical corpus with a 50,000-word vocabulary and 100,000 documents, the full representation matrix has $50,000 \times 100,000 = 5$ billion cells. Storing this as a dense NumPy array of 32-bit floats would require 20 gigabytes of memory — exceeding the RAM of most workstations.

The solution is sparse matrix storage. In practice, any given document contains only a tiny fraction of the total vocabulary — typically 100 to 1,000 distinct tokens out of 50,000. This means more than 98% of the matrix cells are zero. Sparse matrix formats such as `scipy.sparse.csr_matrix` (Compressed Sparse Row) store only the non-zero values and their indices, reducing memory consumption from gigabytes to megabytes. The memory requirement scales with the total number of non-zero entries rather than the product of document count and vocabulary size. `sklearn`'s `CountVectorizer` and `TfidfVectorizer` return `scipy` sparse matrices by default for this reason.

BoW Sparsity — Worked Example

Corpus : 10,000 news articles

Vocabulary: 50,000 words (after `min_df=5` filtering)

Dense matrix size : $10,000 \times 50,000 = 500\text{M cells} \times 4 \text{ bytes} = 2 \text{ GB}$

Average document length : 300 tokens

Average unique words/doc : ~ 200 (many repeated tokens)

Non-zero entries : $10,000 \times 200 = 2,000,000$

Sparse storage (CSR) : $2\text{M} \times (4+4) \text{ bytes} \approx 16 \text{ MB}$ (125× compression)

Sparsity ratio : $2,000,000 / 500,000,000 = 0.4\%$ (99.6% zeros)

6.2 Vocabulary Size and Dimensionality

Raw vocabulary size in a large corpus can reach hundreds of thousands of tokens, including many hapax legomena — words that appear only once in the entire corpus. These rare tokens are statistically uninformative: a model cannot learn a reliable representation of a token it has seen only once, because there

is no way to distinguish whether its single occurrence is representative of its true distributional behaviour or an anomaly. Including hapax legomena in the vocabulary increases dimensionality without contributing predictive power.

The primary hyperparameters for vocabulary control in sklearn's CountVectorizer are `min_df`, `max_df`, and `max_features`. `min_df` specifies the minimum document frequency (number of documents a token must appear in to be included in the vocabulary); setting `min_df=5` removes all tokens appearing in fewer than 5 documents, which typically removes the long tail of typos, rare names, and topic-specific jargon while retaining the majority of informative vocabulary. `max_df` specifies the maximum document frequency as a proportion (e.g., `max_df=0.95` removes tokens appearing in more than 95% of documents) or as an absolute count — a data-driven alternative to explicit stop word lists. `max_features` limits the vocabulary to the top-N most frequent terms after all other filtering.

The interaction between these hyperparameters and downstream model performance is non-trivial and requires empirical tuning via cross-validation. Setting `min_df` too high removes rare but potentially discriminative domain-specific terms; setting it too low includes statistical noise. Setting `max_features` too low throws away vocabulary that may be needed for distinguishing minority classes. The optimal settings depend on the corpus statistics, the task, and the downstream model: regularised models (SVM, logistic regression with L1/L2 penalty) are more robust to larger vocabularies because regularisation suppresses the weights of uninformative features, while unregularised models (Naive Bayes) benefit more from aggressive vocabulary reduction.

Term Frequency-Inverse Document Frequency (TF-IDF) is a weighting scheme that addresses the most significant weakness of raw count BoW vectors: the conflation of frequency with importance. In a raw count vector, a word that appears 50 times in a document receives a count of 50 regardless of whether it is the key topical term of the document or a common grammatical function word. TF-IDF resolves this by scaling the term frequency by a factor that is inversely proportional to how many documents in the corpus contain that term: words that are frequent in a specific document but rare across the corpus receive high weights; words that are frequent everywhere receive low weights.

The intuition behind TF-IDF is rooted in the statistical notion of specificity. A term that identifies a document as belonging to a particular topic or domain should appear more frequently within documents of that type than across the corpus as a whole. Technical terms in a medical abstract — 'myocardial', 'infarction', 'angioplasty' — appear frequently in cardiology documents but rarely elsewhere, making them highly discriminative. General vocabulary terms — 'the', 'study', 'patient' — appear frequently across all medical documents and are therefore poor discriminators within the medical domain. TF-IDF assigns high weights to the former and low weights to the latter.

7.1 Term Frequency and Inverse Document Frequency

The formal definition of TF-IDF requires careful attention to its two components and their combination. Term Frequency $TF(t,d)$ is most simply defined as the raw count of term t in document d . Inverse Document Frequency $IDF(t)$ is defined as the natural logarithm of the ratio of the total number of documents N to the number of documents containing term t : $IDF(t) = \log(N / df(t))$, where $df(t)$ is the document frequency of term t . The logarithm is critical: it ensures that a term appearing in 10% of documents does not receive an IDF weight 10 times that of a term appearing in 100% of documents — the ratio is compressed to a more moderate scale.

TF-IDF — Formal Definition and Worked Example

$TF(t, d)$ = count of term t in document d

$IDF(t)$ = $\log(N / df(t))$

$TF-IDF(t,d) = TF(t,d) \times IDF(t)$

Example: $N=10,000$ documents

Term 'algorithm': $TF=5$ in doc d , $df=200$ documents contain it

$IDF('algorithm') = \log(10000/200) = \log(50) \approx 3.91$

TF-IDF = $5 \times 3.91 = 19.56$

Term 'the': $TF=50$ in doc d , $df=9999$ documents contain it

$IDF('the') = \log(10000/9999) \approx 0.0001$

TF-IDF = $50 \times 0.0001 = 0.005$ (effectively zero)

Result: 'algorithm' (rare, informative) $\gg$ 'the' (universal, uninformative)

The log in the IDF formula also has the effect of making $IDF(t) = 0$ for a term that appears in every document ($df(t) = N$), which automatically suppresses universal terms without requiring an explicit stop word list. This is mathematically elegant but has a practical caveat: the zero-IDF behaviour applies only to terms present in every training document, not to terms that are common across domains but not present in every document of the specific corpus. Domain-specific high-frequency terms that fall just below the threshold (appearing in, say, 95% rather than 100% of documents) will still receive non-trivial IDF weights. Explicit stop word removal remains useful as a complement to TF-IDF weighting.

7.2 Variants: Sublinear TF, Smooth IDF

The raw term frequency $TF(t,d) = \text{count}(t,d)$ has a known deficiency: it scales linearly with the number of occurrences, implying that a term appearing 100 times in a document is 100 times as important as a term appearing once. This linear relationship is empirically unjustified: a term repeated 100 times is certainly more prominent than one appearing once, but the relationship is better described as logarithmic — each additional repetition beyond the first adds diminishing marginal information about the document's topicality.

Sublinear TF scaling addresses this with the transformation $TF_sublinear(t,d) = 1 + \log(\text{count}(t,d))$ if $\text{count}(t,d) > 0$, else 0. This compresses the term frequency scale so that a term appearing 100 times receives a TF of $1 + \log(100) \approx 5.6$ rather than 100, while a term appearing once receives a TF of $1 + \log(1) = 1$. The ratio between high-frequency and low-frequency terms is thus reduced from 100:1 to approximately 5.6:1. sklearn's `TfidfVectorizer` implements this as the `sublinear_tf=True` parameter and consistently improves performance on classification tasks — it is a recommended default for production systems.

Smooth IDF adds 1 to both the numerator and denominator of the IDF formula: $IDF_smooth(t) = \log((N+1) / (df(t)+1)) + 1$. This serves two purposes: it prevents division by zero for terms not present in the training vocabulary (which would cause $df(t) = 0$ in the raw formula), and it ensures that terms appearing in every document receive an IDF of 1 rather than 0, preserving a modest positive weight for universal terms rather than completely suppressing them. sklearn uses smooth IDF by default (`smooth_idf=True`), and this is the appropriate default for most applications.

7.3 TF-IDF in Production Search Engines

TF-IDF was the dominant relevance ranking algorithm for commercial search engines throughout the 1990s and early 2000s. The classic TF-IDF formulation was refined by the information retrieval community into the BM25 (Best Match 25) algorithm (Robertson & Spärck Jones, 1988; Robertson et al., 1994), which addresses two additional limitations of raw TF-IDF: the lack of document length normalisation and the unbounded growth of term frequency contribution.

BM25 introduces two parameters, k_1 and b , that control term frequency saturation and document length normalisation respectively. The k_1 parameter (typically set to 1.2–2.0) controls the point at which additional occurrences of a term contribute negligibly to the relevance score — addressing the same concern as sublinear TF scaling but with a more principled functional form. The b parameter (typically set to 0.75) controls how strongly document length is penalised: a term appearing 5 times in a short 100-word document should be weighted more highly than the same term appearing 5 times in a long 2,000-word document, since it represents a higher proportion of the shorter document's content. BM25 has been the default relevance algorithm for Elasticsearch and Apache Solr since 2015, powering the majority of enterprise search deployments worldwide.



Research Spotlight: Neural Reranking

Modern production search systems such as Google Search, Bing, and Amazon Product Search use a two-stage retrieval architecture: BM25 retrieves a candidate set of 100–1,000 documents efficiently, then a neural reranker (typically a BERT-based cross-encoder) scores each candidate against the query to produce the final ranking. This hybrid approach combines BM25's efficiency and interpretability with BERT's ability to capture semantic relevance and query intent. Pure neural retrieval (dense passage retrieval) is increasingly competitive but remains computationally more expensive at retrieval time.

The Bag of Words model and TF-IDF weighting, despite their effectiveness, share a fundamental limitation: they treat documents as unordered collections of tokens, discarding all information about word order, sentence structure, and higher-level linguistic organisation. This limitation becomes material for tasks where local word order is semantically important — sentiment analysis (where 'not good' vs. 'good not' have very different meanings), syntactic classification, and any task requiring understanding of grammatical relationships between words. Advanced feature engineering techniques address this limitation through various mechanisms for incorporating structural and contextual information into the feature representation.

8.1 N-Gram Features

N-gram features extend the BoW vocabulary to include contiguous sequences of n tokens, capturing local word order patterns that unigram BoW cannot represent. Bigrams are pairs of adjacent tokens; trigrams are triples. The vocabulary of an n -gram model grows approximately as $|V|^n$ in the worst case, though the actual growth is much smaller in practice because most word combinations do not occur and are excluded by minimum document frequency filtering.

The practical value of bigram features is well-established empirically. Adding bigrams to a unigram BoW classifier on sentiment analysis tasks typically yields 2–5% absolute accuracy improvement. The intuitive reason is that several important sentiment patterns are bigram-level phenomena: 'not good', 'not bad', 'very disappointed', 'highly recommended', 'not worth', and 'would not' are all sentimentally informative bigrams that unigram models must reconstruct from individual word co-occurrences — an indirect and less reliable inference. Preserving these as explicit features makes the pattern detection direct and reliable.

The vocabulary explosion problem is the primary practical constraint on n -gram features. A vocabulary of 30,000 unigrams produces approximately 900 million possible bigrams, of which a small fraction will appear with sufficient frequency to be informative. Aggressive minimum document frequency filtering ($\text{min_df}=5$ or higher) is essential to keep the bigram vocabulary tractable. In practice, the number of informative bigrams in a typical corpus is 2–5 times the number of informative unigrams — a manageable expansion. Trigrams and higher-order n -grams rapidly encounter diminishing returns beyond the vocabulary cost, and are used primarily for specific tasks like language identification and character-level modelling where higher-order statistics are informative.

8.2 Character N-Grams for Robustness

Character-level n -grams — overlapping sequences of n characters extracted from the raw text string — provide a complementary feature representation to word-level n -grams with several important practical advantages. Character n -grams are inherently robust to spelling errors and typos, because a misspelling of a word like 'recieve' for 'receive' shares most of its character 3-grams ('rec', 'eci', 'eiv', 'ive') with the correctly spelled form, even though it is a different token at the word level. This property makes character n -grams particularly valuable for NLP on social media text, user-generated content, and any domain with high orthographic variability.

fastText, developed by Facebook AI Research (Bojanowski et al., 2017), uses character n -gram embeddings as the core of its word representation. Rather than learning a single embedding vector for each word token, fastText represents each word as the sum of the embedding vectors of all its character 3-grams to 6-grams. This allows fastText to construct a reasonable representation for any word — including words never seen during training — by composing it from its character subsequences. 'uncharacteristically', for example, can be represented as the sum of the embeddings of 'unc', 'nch', 'cha', 'har', 'ara', 'rac', ..., 'ally', including many n -grams that also appear in related words like 'characterise', 'characteristic', and 'characteristically'. This morphological compositionality enables fastText to generalise to out-of-vocabulary words in a principled way.

For language identification — determining which language a text fragment is written in — character n-grams are the standard feature representation, consistently outperforming word-level features. The reason is that languages have characteristic character n-gram frequency distributions that are largely independent of the vocabulary: a character trigram like 'sch' is strongly indicative of German regardless of whether the specific word containing it appears in the training vocabulary. The fastText language identification model, trained on character n-gram features over 176 languages, achieves near-perfect accuracy on clean text in under 1 millisecond of inference time, making it the standard tool for language detection in multilingual NLP pipelines.

8.3 Linguistic Features: POS, Dependency, NER

While the BoW and n-gram features discussed above are purely distributional — derived entirely from the statistical co-occurrence patterns of tokens in text — a second class of features derives from linguistic analysis of the text's grammatical structure. These linguistic features capture structural properties of text that distributional features cannot represent: the grammatical roles of words (subject, object, modifier), the types of entities mentioned, the syntactic complexity of sentences, and the distribution of grammatical categories.

Part-of-speech (POS) distribution features encode the proportion of each grammatical category in a text: the noun-to-verb ratio, the density of adjectives, the frequency of modal verbs ('should', 'could', 'must'). These features are informative for genre and register classification: academic writing has a characteristic POS distribution (high noun density, low pronoun density, high complex-NP density) that distinguishes it from journalistic writing, which in turn differs from conversational text. POS features are also the primary input to readability metrics (Flesch-Kincaid, Gunning Fog) that quantify text complexity.

Named entity type distribution features — the count of PERSON, ORGANISATION, LOCATION, DATE, and MONEY entities per document — are powerful signals for document genre and topic. A document mentioning many PERSON and LOCATION entities is likely a news article about a specific event; a document mentioning many MONEY and ORGANISATION entities is likely a financial document; a document with high DATE density may be a historical analysis or a timeline. These features are particularly valuable in low-resource settings where large pre-trained language models are unavailable or computationally prohibitive, providing structured semantic signal that raw token features cannot capture.

Dependency structure features go beyond the token level to encode relationships between tokens: subject–verb pairs, verb–object pairs, modifier–head pairs. These relational triples are the foundation of open information extraction (OpenIE) systems that extract structured facts from text. For classification tasks that require understanding of who did what to whom — event detection, cause-effect extraction, medical relation extraction — dependency features provide structural signal that neither BoW nor n-gram features can capture. spaCy provides dependency parsing as a built-in pipeline component, making these features accessible with minimal implementation effort.

The transformer era has profoundly changed the relationship between preprocessing and model performance. BERT, RoBERTa, and their successors are trained end-to-end on raw (or lightly cleaned) text using subword tokenisation, and their internal representations implicitly encode many of the normalisations that classical preprocessing pipelines implement explicitly. A BERT model fine-tuned on text that has been aggressively stemmed and had stopwords removed will typically perform worse than one fine-tuned on lightly preprocessed text, because the aggressive preprocessing destroys contextual information that BERT's attention mechanism is specifically designed to exploit.

This does not mean preprocessing is irrelevant for transformer-based systems. Three distinct scenarios maintain the practical importance of classical preprocessing decisions. First, low-resource languages without available pre-trained transformer models continue to rely on classical pipelines: morphological segmentation, stemming, and stop word removal remain the primary mechanisms for vocabulary reduction and feature engineering when a 100M-parameter pre-trained model is not available. Second, domain-specific corpora in clinical, legal, and scientific domains often present distribution shifts severe enough that general-domain pre-trained models underperform carefully engineered classical features, particularly for low-data fine-tuning scenarios. Third, production systems with strict latency or memory constraints — edge deployment, embedded systems, or services requiring sub-5ms response times — may require the much smaller model footprint and lower inference cost of TF-IDF + linear classifier pipelines.

An emerging research perspective blurs the traditional distinction between preprocessing and model training: data augmentation during preprocessing can be viewed as a form of learned preprocessing. Wei and Zou (2019) demonstrated that four simple text augmentation operations — random synonym replacement, random insertion of synonyms, random token swap, and random token deletion — applied stochastically during training can double the effective training set size and improve accuracy on low-resource text classification tasks by 1–5%. This finding has catalysed a broader research programme on text data augmentation, including back-translation (translating to another language and back), paraphrase generation, and mixup interpolation of sentence embeddings.



Research Spotlight: Neural Preprocessing — Wei & Zou (2019)

EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks demonstrated that random deletion, swap, insertion, and synonym replacement during preprocessing can double effective training set size and improve robustness on low-resource classification tasks — blurring the line between preprocessing and data augmentation. The technique requires no external data sources, is language-agnostic, and can be implemented in fewer than 50 lines of Python. Subsequent work has extended this to back-translation augmentation, conditional text generation for minority class oversampling, and embedding-space interpolation (mixup), establishing data augmentation as a mainstream preprocessing technique for low-resource NLP.

Looking forward, the distinction between preprocessing and representation learning is likely to continue eroding. The end-to-end training paradigm — in which the model learns to attend to relevant input features from raw text without hand-engineered preprocessing — is the dominant direction of the field. However, the practitioner insights encoded in classical preprocessing pipelines — about which surface features carry signal, which introduce noise, and how domain-specific vocabulary behaves — will remain valuable as inductive biases for system design, training data curation, and diagnostic analysis of model behaviour, even as the explicit preprocessing steps themselves become less necessary for state-of-the-art performance.

The future of AI is bright, and you can be part of it!